

Predicting Container Memory Demand from Kernel Pressure-Stall Signals: PSI-Driven Models for Proactive cgroup Sizing

Course 236502 — Project in Artificial Intelligence, Spring 2025/2026 Advisor: Assaf Singer · Lecturer: Prof. Shaul Markovitch · Revised (v2) following the advisor's feedback, July 2026.

Participant	Student ID	Email
Abed Agbaria	213538887	abedh2003@gmail.com
Ramez Dahle	212763239	ramez.dahla@campus.technion.ac.il
Kameel Zaid	213969660	kameel.zaid@campus.technion.ac.il

(Three-member group — extended scope: a baseline ladder with a PSI-ablation study at every rung, plus a closed-loop control evaluation against three baselines.)

1. The Problem

Every container in a data center runs under a memory limit (a `cgroup` — control group — setting called `memory.max`). Setting this limit is a hard trade-off: set it too **low** and the container is killed by the kernel's OOM (Out-Of-Memory) killer; set it too **high** and memory is wasted, reducing how many containers fit on a machine. Production systems such as Google Autopilot and Kubernetes VPA (Vertical Pod Autoscaler) choose the limit from recent **raw memory usage** plus a safety margin.

The problem we attack is that **raw usage is an ambiguous signal**. A container sitting at 90% of its limit may be perfectly healthy — if that memory is file-backed page cache, the kernel can reclaim it instantly at no cost. The *same* 90% may instead be a program leaking anonymous heap that cannot be reclaimed and is seconds away from an OOM kill. To a model that sees only raw usage, these two situations are identical, yet they require opposite actions.

We therefore define the following **prediction problem**: given a short sliding window of a container's recent runtime metrics, predict its **peak memory demand over the next control horizon** (30–60 s), so that the limit can be adjusted *proactively* — raised before a genuine shortage causes an OOM kill, and lowered when headroom is provably safe to reclaim (§2 justifies this target). Success means **low prediction error on unseen runs, few OOM kills, and little wasted headroom, simultaneously**.

Our key idea is to add a signal that directly disambiguates the cases above: **Linux PSI (Pressure Stall Information)** — a kernel metric reporting the fraction of wall-clock time in which tasks were *stalled* waiting for memory. It is ~0 when all resident memory is evictable cache, and rises only when the kernel is genuinely struggling (swap input/output, direct reclaim).

Container state	Raw usage	PSI	Correct action
Page cache from file reads	HIGH	0	Do nothing — cache is free to evict
Anonymous heap leak	MEDIUM	> 0	Raise the limit
Burst workload, mid-burst	HIGH	> 0	Raise the limit
Burst workload, idle phase	LOW	0	Lower the limit safely

Raw usage confuses rows 1–2 and misses row 4; PSI separates all four.

1.1 Prior approaches — and why PSI has not been used for prediction

Existing systems are all *reactive*. Google Autopilot [EuroSys 2020] sizes memory from moving-window statistics (window peak / high percentile); notably, its *ML-based* recommenders cut memory limits too aggressively and caused OOM kills, so production fell back to the conservative statistic — evidence that raw usage alone is not safe enough for learned memory sizing. Kubernetes VPA keeps decayed usage histograms and applies them by evicting the pod. Meta's Senpai [TMO, ASPLOS 2022] and Kondense are the closest works: they use **PSI itself as a reactive feedback signal**, squeezing `memory.high` until stalls appear — proving PSI is production-grade, but they must *cause* stalls to size the container, and react only after pressure materializes. The only ML-on-PSI work we found, vPALs [AAAI-24 workshop, Huawei], predicts application *latency* from machine-level CPU/IO PSI, discards memory-PSI (its workloads were not memory-constrained), and released no data. **No published system predicts memory demand from PSI:**

	Usage signal	PSI signal
Reactive	Google Autopilot, Kubernetes VPA	Senpai, Kondense
Predictive	LSTM-sizing papers (our ablation baseline)	this project

Why is the quadrant empty? PSI is **recent** (kernel 4.20, Dec 2018; per-container PSI needs cgroup v2, a default only since ~2021–23); it is **absent from every public dataset** (§3 — it cannot be reconstructed from usage traces); and the **actuation path only just matured** — Kubernetes gained restart-free in-place pod resizing (v1.35 GA, Dec 2025) and GA PSI metrics (v1.36, May 2026), making PSI-driven prediction deployable for the first time.

2. Proposed Approach

Mapping to a problem type from class. This is a **supervised learning** task, specifically **time-series regression (sequence-to-value forecasting)**: each example will be a sliding window of a container's recent metric samples, and the target a continuous value describing its upcoming memory demand.

Prediction target. The operational task is *limit assignment*, and a limit must cover the **peak** of the coming interval, not the next sample. We will therefore predict the **maximum memory usage over the next control horizon** (tens of seconds), and derive the assigned limit from the prediction plus a safety margin. Next-step usage will be kept only as a sanity check, since it is nearly solved by trivially repeating the last value; OOM risk needs no separate model, because an OOM occurs exactly when the actual peak exceeds the assigned limit — it becomes an evaluation metric of the peak predictor (§4).

Feature engineering — the core of the work. Following the course's emphasis on *defining informative features from multiple sources rather than using a ready-made table*, each timestep will combine two kernel subsystems: **PSI features (novel)** — the "some" (≥ 1 task stalled) and "full" (all tasks stalled) pressure at the kernel's three averaging horizons; and **memory features (conventional baseline)** — current usage, the current limit, their ratio, and the recent change in usage.

Modeling path — simple baselines before the LSTM. Rather than committing to a recurrent network from the start, we will evaluate models in increasing order of complexity, moving up only when the added complexity earns its keep:

1. **Persistence** — predict that the target equals its current value; the floor every model must beat.
2. **Tabular model on windowed features** — summarize each signal over the window (mean, max, slope, variance, recent delta) and train Random Forest / XGBoost regressors; their feature importances will also show directly how much the PSI features contribute.
3. **LSTM on raw sequences** — a recurrent network that can learn temporal dynamics beyond hand-made aggregates (leaks trend upward, bursty workloads oscillate); justified only if it outperforms the tabular model.

How we exploit the extra information. Our central hypothesis is that PSI improves predictions precisely at the onset of genuine pressure. We will test this with an **ablation at every rung**: each model trained once with the PSI features and once without them, and all of them compared against a **percentile heuristic baseline** in the style of Google Autopilot's sizing rule. Any gap between the two variants is attributable to the PSI features alone; if a simple model with PSI beats a complex one without it, that is itself a positive result — the claim is about the *signal*, not the architecture.

3. System Description

We plan to build a pipeline of the following components, implementing the application logic ourselves and using external libraries only as infrastructure.

- **Data collection.** A component that will run several Docker workloads and sample PSI and memory statistics from each container's `cgroup` (control group) files into a labeled dataset. *Why collect our own data:* we examined the public traces — Google (2011, 2019), Alibaba (2017, 2018, GPU-2023), Azure, Bitbrains, and the ACM Computing Surveys 2025 survey of public cloud datasets — none contains PSI, and none can: PSI is computed live from kernel scheduler stall events, and two containers with identical usage curves can have opposite PSI (the very ambiguity of §1), so it cannot be reconstructed from recorded usage. Each workload will be run **several independent times with varied parameters** so that entire runs can be held out for testing (§4), and a **trace-replay** workload will replay real memory-demand shapes from the public traces inside our instrumented containers — real dynamics, live PSI.

Workload	Behavior	PSI expected
memory leak	steadily allocates and never frees, under a tight limit with swap	rising
file-read burst	repeatedly reads a large file into page cache	none
steady state	holds a small fixed working set	none
bursty batch	alternates busy and idle phases under a tight limit with swap	oscillating
trace replay	replays real memory-demand shapes from public traces	workload-dependent

- **Environment validation & monitoring.** A component to confirm PSI is readable per-container, and a live dashboard to observe metrics during collection.
- **Model training.** A component that will build the sliding windows and windowed aggregates, normalize the features, train each model of §2, and store the trained models. Data will be split **by run** — test runs never seen in training — to prevent look-ahead and within-run leakage.
- **Closed-loop controller.** The online part: it will run a live container, feed the rolling window to the trained model each step, and write the predicted demand (plus a safety margin) back to the container's memory limit. Safety rules will prevent shrinking below live usage and excessive rewrites. Interchangeable modes — our model, the percentile heuristic, **Senpai (Meta's reactive-PSI tool)**, and a fixed limit — will allow a direct comparison on the same container.
- **Evaluation.** A component that scores all methods on identical held-out data and produces the figures.

External components (infrastructure only): PyTorch (LSTM), scikit-learn and XGBoost (tabular baselines, scaling, hyper-parameter search), NumPy/Pandas (data), Matplotlib (plots), Docker Engine (container control through `cgroup v2`), the Linux PSI kernel interface, and a terminal-UI library for the dashboard. Everything else will be written by us.

4. Planned Experiments

Dataset. The labeled workloads described above: several independent runs per workload with varied parameters, plus trace-replay runs, split **by run** into training, validation, and test sets.

Metrics. *Prediction quality:* **MAE** and **RMSE** between predicted and actual peak memory, per workload, averaged over test runs and always reported alongside the persistence baseline, so that skill is measured against the trivial forecaster. *Operational quality (what matters in production):* **wasted headroom** — the average gap between the assigned limit and actual usage — and **OOM rate** — how often actual demand exceeds the assigned limit. Both depend on the safety margin, so we will report them jointly as a **trade-off curve across safety margins** rather than at a single arbitrary setting.

Experiment series. 1. *Main ablation — does PSI help?* Compare the with-PSI and without-PSI variants at every rung of §2, and all of them against the percentile heuristic, overall and per workload; tree feature importances give a second, interpretable line of evidence. We expect the PSI benefit to concentrate on workloads where genuine pressure occurs and to be small where memory is evictable cache — a pattern that would itself support the hypothesis. 2. *Generalization* (essential since we generate the data ourselves): (a) **held-out runs** — entire runs never seen in training; (b) **parameter shift** between training and test runs; (c) **leave-one-workload-out** — train on three workload types, test on the fourth; (d) **trace-replay** runs as a semi-real unseen test set. 3. *Closed-loop control:* run the controller in each mode on live containers and compare wasted headroom and OOM events end-to-end, not only offline error. 4. *Error distribution:* examine the spread of per-step errors with a CDF, to see *when* each method fails, not only its average.

Hyper-parameter tuning (on the training set only). We plan to tune the history-window length, the prediction horizon, the tree models' capacity, the network's size and depth, the learning rate, and the controller's safety margin, using a validation split and grid search, keeping the test set untouched until the final evaluation.

5. Technical Details

- **Language / runtime:** Python 3.11 on WSL2 (Ubuntu, kernel ≥ 4.20 for PSI) with Docker Desktop.
- **Learning:** PyTorch (LSTM); scikit-learn + XGBoost (tabular baselines, scaling, grid search).
- **Data / plotting:** NumPy, Pandas, Matplotlib, Rich.
- **Systems interfaces:** Docker Engine + `cgroup v2` pseudo-files (`memory.pressure`, `memory.current`, `memory.max`; `memory.high` for the Senpai baseline).
- **Reproducibility:** each stage is a standalone script driven from the collected dataset (CSV) and the saved model, documented in a `README`.

6. Use of Generative AI

Generative-AI tools may be used, per course policy, to assist with code writing and for language polishing of this document; all methodology, experiments, and analysis are our own, and a full declaration will be included in the final report.